

NYC 311 Service Request Analysis:

Dataset Curation, Supervised Learning, and Unsupervised Discovery

The timestamp difference calculated to determine how long does a city take to close a service complain is a quantifiable measure of government performance. This analysis measures that performance by focusing on whether the outcome of each complain through New York City's 311 service request can be predicted during filing. The portal has served as city's central intake point for non-emergency complaints from residents, logging millions of records across all five boroughs each year. The system covers a wide range of categories, including noise violations, heat outages, blocked fire hydrants, and dozens of others, each tied to a specific agency and resolution process.

A noise complaint filed at midnight in the Bronx enter the same system as a heating failure reported at 9 AM in Manhattan, but they route to different agencies, carry different urgency levels, and close on different timescales. Each record stores a creation timestamp, a closure timestamp, the complaint type, the borough of origin, a community board identifier, and the submission channel. For the analysis, the data was limited to 50,000 rows, providing a working sample large enough to support stratified splitting and feature engineering. The prediction target, `closed_within_7_days`, was constructed from the difference between creation and closure timestamps. Any complaint that closed within 168 hours received a class label of 1; anything beyond that threshold received a 0. Only records with a status of Closed and a non-negative response time qualified for the final dataset. After applying those filters, 49,458 rows remained, of those, 82.2 percent were class 1 and 17.8 percent were class 0. This distribution reflects how the 311 system works: most complaints get resolved within a week, but the minority

that do not are randomly distributed. Identifying what separates fast resolutions from slow ones was the central question driving this analysis.

Data Cleaning and Feature Engineering

StratifiedShuffleSplit divided the 49,458 qualifying rows into 39,566 for training and 9,892 for testing at an 80/20 ratio. Stratifying on the target kept both sets at the same 82.18/17.82 class split. The pipeline fit all preprocessing parameters on the training set only and applied the same transformations to the test set without letting it contribute to any estimates. The raw data had 46 columns, most of them unusable. The pipeline dropped a large group that had more than 90 percent missing values and removed a second group because those columns stored information that only becomes available after a complaint closes, meaning any model trained on them would read the outcome rather than predict it. At the row level, the pipeline excluded records with unrecognized borough values and nulled any closure timestamp that preceded the creation timestamp rather than dropping the row entirely.

From the remaining columns, the feature engineering step constructed seven variables to capture the geographic context of each complaint: when it arrived, what day, whether the resident used the web portal, and which community board area the complaint came from. These choices reflected the expectation that filing time and submission channel might relate to how complaints get routed and prioritized. A median imputer fit on the training set filled the remaining missing values. The median outperformed the mean here because response time was heavily right-skewed, and mean imputation would have pushed estimates far above what most records showed. One-hot encoding converted the categorical columns into binary dummy variables bringing the final feature count to 17.

Data Visualization

The histograms showed complaints concentrating heavily in daytime hours and on weekdays, with a sharp drop on evenings and weekends. Response time stood out immediately: while most complaints resolved within a few hours, a small tail stretched to over 33,000 hours and pulled the mean to 628 hours against a median of just 25. The class imbalance was equally visible, with fast resolutions outnumbering slow ones by nearly five to one. The scatter matrix plotted response time against the target showed: fast complaints clustered near zero and slow ones sat far to the right.

The transformation analysis tested five reshapings of response time and filing hour to determine whether restructuring the distributions would improve their output. For response time, squaring and cubing amplified the existing skew, compressing most values near zero while pushing the outliers further away. The log transform spread the distribution across a wider range exposing two peaks: one cluster of complaints resolving within an hour and a second taking a few days. The exponential offered no improvement over the original. Based on this comparison, the log transform is the appropriate reshaping for response time. For filing hour, the original bell-shaped distribution provided a good pattern. Squaring and cubing shifted weight toward later hours without adding useful structure, and the log stretched the early morning hours while compressing midday hours.

PCA for Feature Selection

The analysis tested two classification models on the prepared data: Logistic Regression and Random Forest. Logistic Regression fits a linear decision boundary between two classes and outputs a probability score between 0 and 1. Complaints scoring above 0.5 classify as fast; those below classify as slow. With default settings, the model scored 88.7 percent on training and 88.8

percent on the test set. The confusion matrix showed the model correctly identified all fast complaints but misclassified 1,105 of the 1,762 slow ones. Five-fold cross-validation returned a mean of 0.882 and confirmed the pattern across every fold. GridSearchCV identified $C=100$ with balanced class weighting as the best configuration, raising the cross-validation score to 0.991. By applying those settings, the model correctly identified 1,730 of the 1,762 slow complaints on the test set and reached 99.3 percent accuracy. The coefficient chart showed response time carrying nearly all the predictive weight, with every other feature registering too small to read at the same scale.

Random Forest trains an ensemble of decision trees on different samples of the data and combines their predictions by majority vote. The default model scored 100 percent on both sets. That result reflects data leakage rather than genuine learning: the target defines at 168 hours of response time, so any tree locates that split immediately and stops there. Cross-validation returned 1.000 across all five folds. GridSearchCV tested 16 combinations of tree count and depth and found that the best configuration matched the default result. Feature importances placed 90 percent of the total weight on response time alone, leaving every other variable near zero. Both models converged on the same shortcut, and no parameter setting could change that.

Clustering with PCA Pre-processing

With supervised learning producing near perfect results, clustering to examined whether the data contained any natural groupings independent of the target label. Three algorithms tested on both raw scaled features and the 11 PCA components, with each run scored on Adjusted Rand Index, and Silhouette Coefficient. K-Means assigns each point to the nearest of k cluster centers, recalculates those centers as the mean of their assigned points, and repeats until assignments stabilize. The elbow method pointed to $k=5$, where adding more clusters stopped reducing inertia

meaningfully. Both the raw and PCA runs produced three clusters on one side and two on the other, with a diagonal stripe pattern cutting through each group, driven by the discrete values of filing hour and day of week. ARI came in at -0.0008, indicating no alignment with the resolution outcome. Silhouette scores reached 0.2189 on raw features and 0.2370 on PCA.

Agglomerative clustering starts with every point as its own group and merges the two closest at each step. Ward linkage guided each merge by selecting the pair whose combination produced the smallest increase in total within cluster variance. The dendrogram showed two tall branches at the top, pointing clearly to $k=2$. Both runs produced one large group and a second smaller one, ARI of -0.0023, Silhouette scores of 0.1669 and 0.1800, lower than K-Means. DBSCAN found clusters by locating dense regions in the feature space. The default eps of 0.5 fragmented the data into hundreds of clusters; raising it to 3.0 produced 22 clusters on raw features and 21 on PCA. The scatter plots showed the same stripe pattern as the other methods. ARI reached 0.0043, Silhouette scores of 0.3392 and 0.3873 were the highest of the three algorithms, but those scores reflect geometrically tight slices of response time rather than any structure that connects meaningfully to the resolution outcome.

Summary

Across all three projects, the results pointed in the same direction, although not toward the conclusions the analysis originally anticipated. The preprocessing pipeline performed as designed: StratifiedShuffleSplit maintained class proportions across both sets, median imputation handled the right-skewed response time distribution more appropriately than a mean-based approach would have, and column alignment after one-hot encoding ensured the training and test sets remained comparable throughout. The hyperparameter search for Logistic Regression

highlighted: pairing $C=100$ with balanced class weighting correctly identified 1,730 of 1,762 slow complaints on the test set, which suggests that class regularization can alter classification behavior when the minority class is the prediction target.

The dominant limitation of the analysis was the inclusion of `response_time_hours` as a predictor. Because the variable derives directly from the closure timestamp that defines the target, both supervised models identified the 168-hour threshold immediately and built their predictions around it. The model predicted fast for nearly every complaint produced 88.8 percent accuracy on the test set during Logistic Regression and the confusion matrix revealed that most slow complaints were never detected. Random Forest avoided that issue by finding the response time threshold regardless of class proportions. None of the three clustering algorithms in unsupervised results produced groupings that aligned with the resolution outcome, and the stripe patterns visible across every scatter plot traced the density structure of `response_time_hours` rather than any meaningful category in the data.

If I had to do the analysis all over again I would remove `response_time_hours` before modeling, as the variable does not predict the target; it defines it. I would add `complaint_type` because the 311 system routes different complaint categories to different city agencies, and those agencies do not operate on the same timelines or under the same resource constraints. The current feature set captured when and where complaints arrived but failed to highlight what kind of problem the resident reported.